



[02_ATmega128 개요]

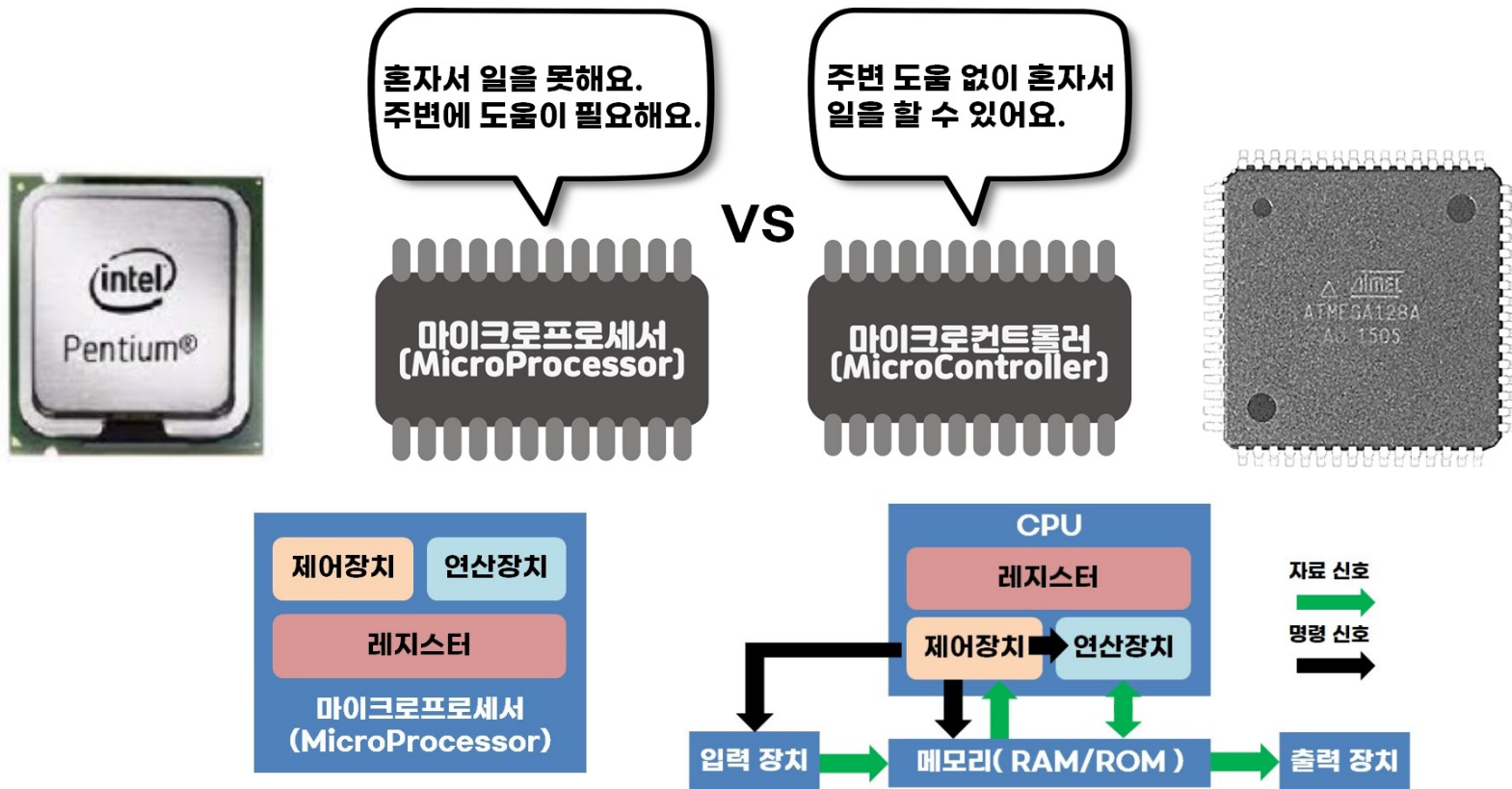
한국폴리텍대학교 성남캠퍼스

1

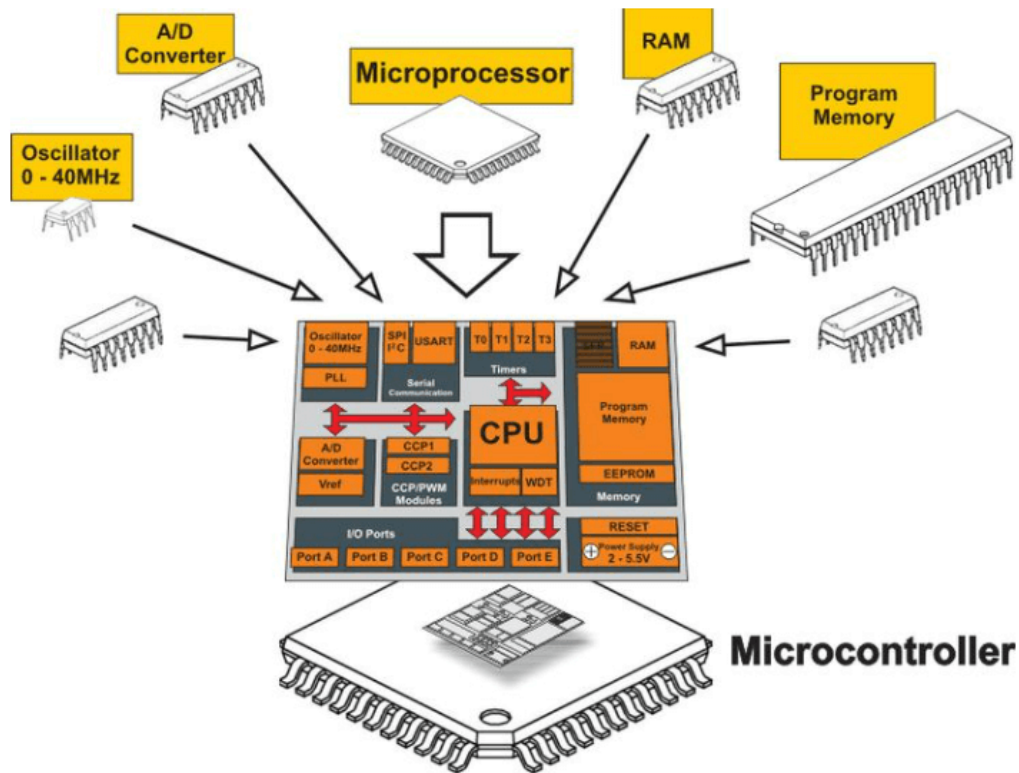
마이크로 컨트롤러

마이크로 프로세서 vs 마이크로 컨트롤러

- ❖ 마이크로 프로세서는 주로 연산처리를 위한 직접회로를 칭하며, 마이크로 컨트롤러는 하나의 칩 안에 마이크로 프로세서와 기본 시스템을 구성할 수 있는 주변 기능(RAM, I/O 시스템 등)을 내장하고 있음



마이크로 컨트롤러



- 임베디드 시스템 (embedded system)에서 주로 사용
- 특정 시스템 전용의 단순한 기능 구현을 위해 사용

마이크로 컨트롤러

❖ 마이크로 컨트롤러의 장점

- 제품의 소형화 경량화
- 저렴한 가격
- 신뢰성 향상
- 융통성

❖ 마이크로 컨트롤러의 단점

- 낮은 처리 능력
- 범용성 부족

❖ 마이크로 컨트롤러의 활용분야

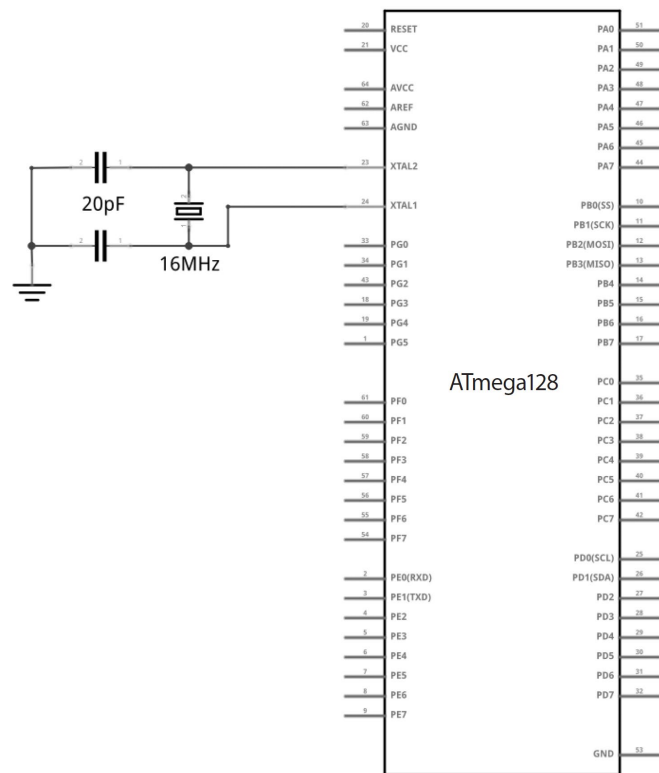
분야	예
의료	의료기 제어, 자동 심박계
교통	신호등 제어, 주차장 관리
감시	출입자/침입자 감시, 산불 감시
가전	에어컨, 세탁기, 전자레인지
음향	CD 플레이어, 전자 타이머
사무	복사기, 무선 전화기
자동차	엔진 제어, 충돌 방지
기타	게임기, 차고 개폐 장치

Atmega128 동작을 위한 기본 회로

❖ 전원만 인가하면 동작 가능하며, 외부에 오실레이터 회로를 구성하여 원하는 속도로 동작시킬 수 있음

- 내부 RC 오실레이터는 1, 2, 4, 8 MHz의 고정 주파수 클럭을 생성
- 외부 오실레이터에 비해 정밀도는 낮음
- CKSEL3 ~CKSEL0 설정에 의해 주파수 결정

CKSEL3..0	주파수(MHz)
0001	1.0
0010	2.0
0011	4.0
0100	8.0



마이크로 컨트롤러의 데이터 교환

❖ ATmega128은 디지털 컴퓨터의 일종

- 0과 1의 이진 값을 기본으로 함
- ATmega128의 핀을 통해 비트 단위의 데이터 교환
- 64개 핀 중 53개 핀을 비트 단위 데이터 교환에 사용 가능
 - ✓ 실제 데이터는 바이트 이상의 단위
 - ✓ 주변장치에 따라 많은 핀이 필요하므로 53개로 부족

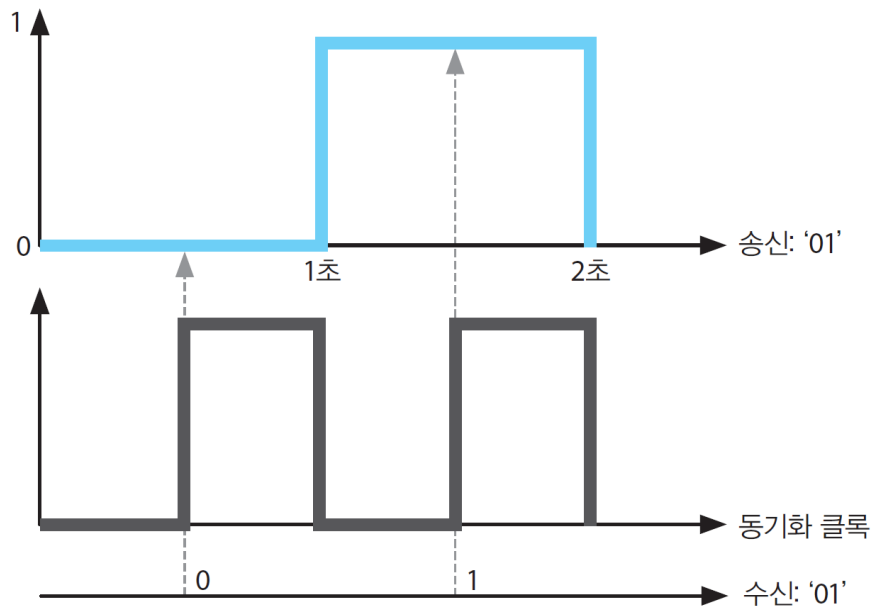
❖ 직렬 vs 병렬 통신

- 병렬 통신 : 여러 개의 핀을 사용, 바이트 단위 이상의 데이터를 동시에 전송
- 직렬 통신 : 적은 수의 핀을 사용, 바이트 단위 이상의 데이터를 여러 번에 나누어 전송
 - ✓ 핀 수가 제한된 마이크로 컨트롤러에서 선호

마이크로 컨트롤러의 데이터 교환

❖ 동기식 직렬통신

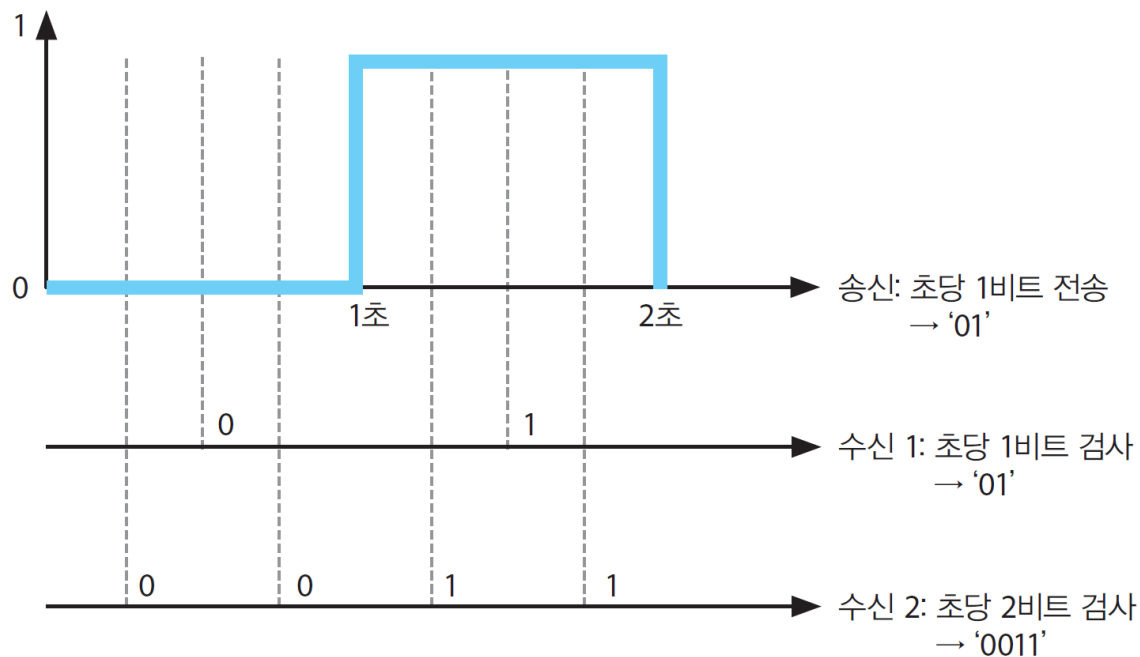
- 여러 번에 걸쳐 전송되는 데이터 비트 구분을 위해 별도의 동기화 클럭 사용
 - ✓ SPI, I2C



마이크로 컨트롤러의 데이터 교환

❖ 비동기식 직렬통신

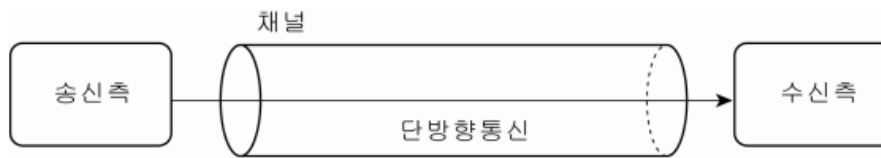
- 전송 속도를 약속하여 사용하고 별도의 클럭을 사용하지 않음
 - ✓ 동일한 속도로 송수신 하지 않으면 잘못된 데이터가 전달
 - ✓ UART



마이크로 컨트롤러의 데이터 교환

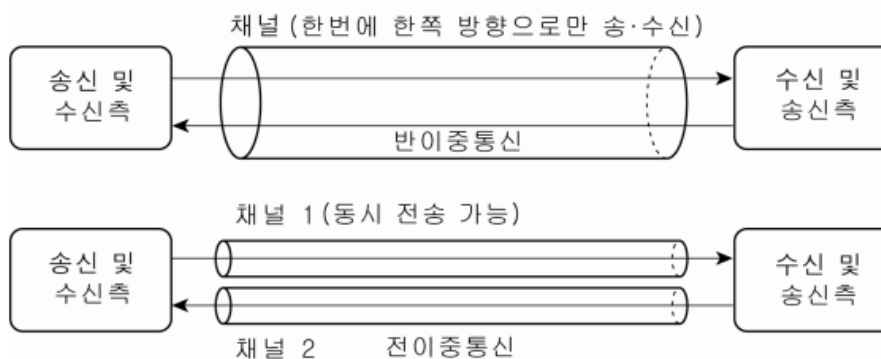
❖ 전 이중 방식(Full Duplex)

- 송수신을 위해 2개의 연결선 사용
- 송수신 동시에 가능



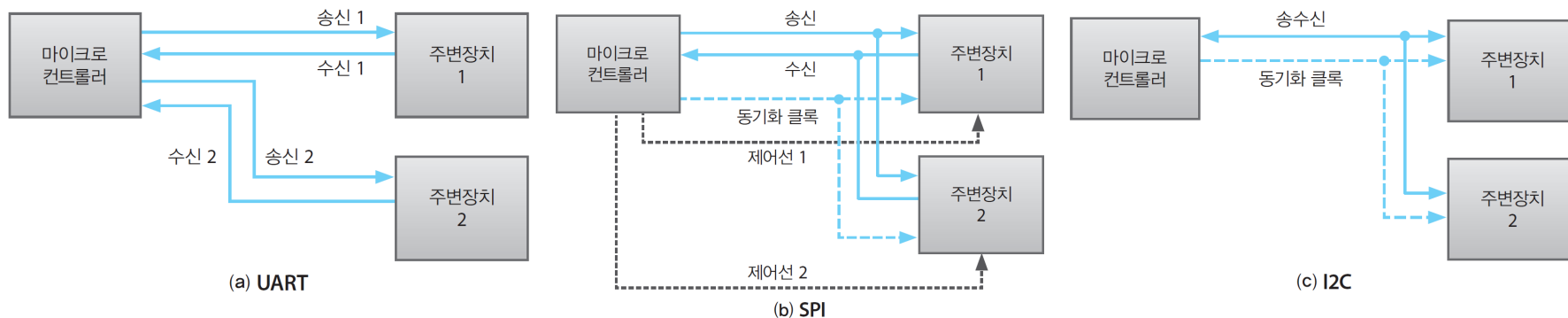
❖ 반 이중 방식(Half Duplex)

- 송수신을 위해 1개의 연결선 사용
- 송수신 동시에 불가능



데이터 선 개수	동기 클럭 사용	동기 방식	비동기 방식
전이중 방식		SPI	UART
반이중 방식		I2C(또는 2-와이어)	1-와이어

마이크로 컨트롤러의 데이터 교환



			UART	SPI	I2C
연결 방법			1:1	1:n	1:n
전송 방법			전이중	전이중	반이중
연결선	1개 슬레이브 연결	데이터	2개	2개	1개
		동기 신호	-	1개	1개
		제어	-	1개	-
		합계	2개	4개	2개
	n개 슬레이브 연결		2n개	(3 + n)개	2개

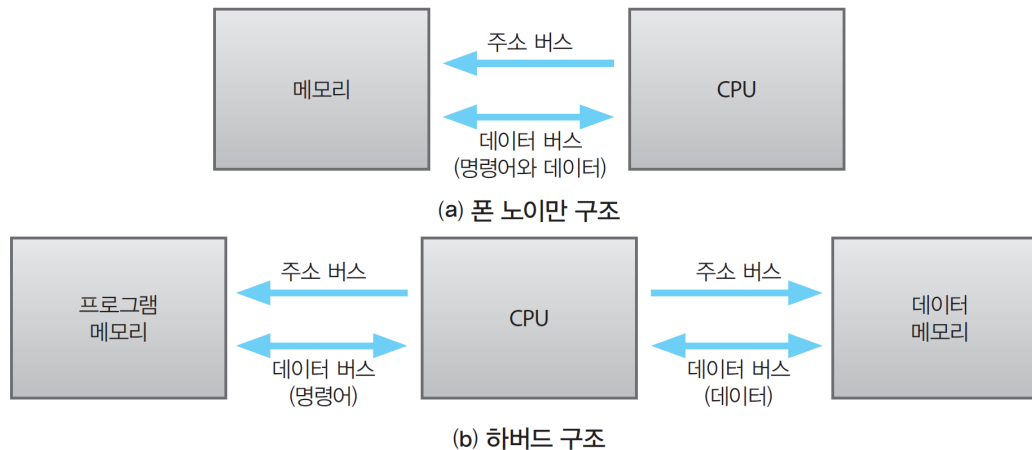
프로세서 구조

❖ 폰 노이만 구조(주로 Intel 계열 CPU)

- 명령어와 메모리가 하나의 메모리에 저장
- 상대적으로 느린 메모리 속도에 의한 병목 현상

❖ 하버드 구조(주로 마이크로 컨트롤러)

- 명령어와 메모리를 전용의 메모리에 나누어 저장
- 하드웨어는 복잡해지지만 병목 현상 해소의 한 방법



Instruction Set Architecture

❖ CISC vs RISC

- 컴퓨터에서 프로세서가 지원하는 명령어 집합과 작동 방식을 정의한 것을 ISA(Instruction Set Architecture), 즉 명령어 집합 구조라고 함
- ISA에 의한 정의 방식에 따라서 컴퓨터 프로세서의 종류는 CISC와 RISC로 구분
- CISC(Complex Instruction Set Computer)는 다양하고 복잡한 명령어를 지원
 - ✓ CISC에서는 한 개의 명령어로 여러 개의 기본 연산을 수행할 수 있도록 다양한 명령어 제공
 - ✓ 복잡한 작업을 단일 명령어로 수행할 수 있으므로 프로그램의 크기를 줄이고 실행 시간을 단축시킬 수 있음
 - ✓ CISC의 가장 대표적인 예는 PC의 CPU로 사용되는 인텔사의 x86 계열의 프로세서
- RISC(Reduced Instruction Set Computer)에서는 단순하고 고정된 길이를 가지는 명령어 지원
 - ✓ RISC 프로세서에서는 파이프라인, 병렬 처리, 예측 실행 등을 통한 성능 향상이 가능
 - ✓ RISC의 대표적인 예는 Arm사에서 설계한 프로세서

Instruction Set Architecture

❖ CISC vs RISC

ADD R1, A, B ; $R1 \leftarrow M[A] + M[B]$
 ADD R2, C, D ; $R2 \leftarrow M[C] + M[D]$
 MUL X, R1, R2 ; $M[X] \leftarrow R1 * R2$

LOAD R1, A ; $R1 \leftarrow M[A]$
 LOAD R2, B ; $R2 \leftarrow M[B]$
 LOAD R3, C ; $R3 \leftarrow M[C]$
 LOAD R4, D ; $R4 \leftarrow M[D]$
 ADD R1, R2 ; $R1 \leftarrow R1 + R2$
 ADD R3, R4 ; $R3 \leftarrow R3 + R4$
 MUL R1, R3 ; $R1 \leftarrow R1 * R3$
 STORE X, R1 ; $M[X] \leftarrow R1$

	CISC	RISC
명령어 개수	많음	적음
프로그램 크기	작음	큼
하드웨어 복잡도	높음	낮음
소프트웨어(컴파일러) 복잡도	낮음	높음
명령어당 클럭 수	가변	고정
전력 소모	많음	적음
호환성	높음	낮음
대표적인 CPU 제조사	인텔	ARM

메모리의 종류

- ROM과 RAM : 전원이 주어지지 않을 때 기록된 내용의 보존 여부로 구분

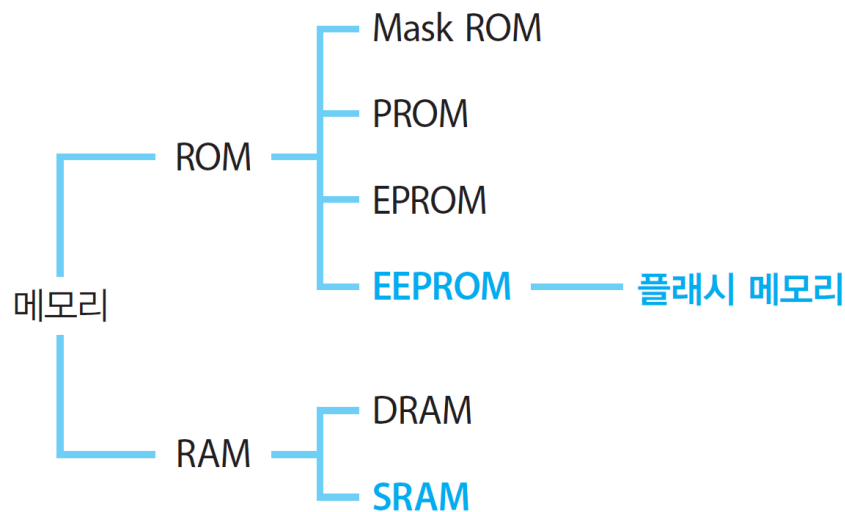
	(쓰기 가능한) 롬	램
읽기 속도	빠름	빠름
쓰기 속도	느림	빠름
휘발성	×	○
수명	제한된 수명	반영구적 수명

- SRAM과 DRAM : RAM을 만드는 방식에 따라 리프레시의 필요 여부로 구분

	SRAM	DRAM
읽기/쓰기 속도	빠름	느림
리프레시	×	○
집적도	낮음	높음
가격	비쌈	저렴

메모리의 종류

- Mask ROM
 - ✓ 생산 시에 내용이 결정되고 내용 변경이 불가능
- PROM (Programmable ROM)
 - ✓ 전용 장치를 통해 한 번만 내용 기록이 가능
- EPROM (Erasable PROM)
 - ✓ 전용 장치를 통해 쓰고 지우기가 가능
- EEPROM (Electrically EPROM)
 - ✓ 전기 신호만으로 쓰고 지우기가 가능
 - ✓ 바이트 단위 읽기/쓰기 지원
- 플래시 메모리
 - ✓ 블록 단위 읽기/쓰기 지원
 - ✓ 대용량 메모리에 적합
 - ✓ 대부분의 휴대용 모바일 기기에서 채택



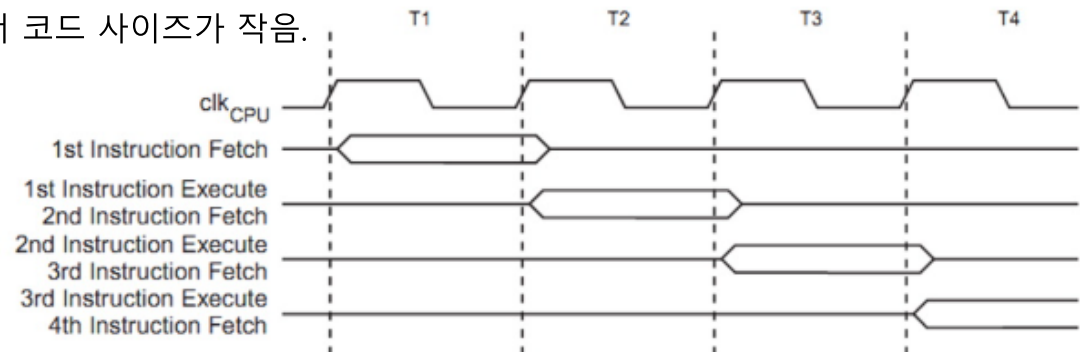
2 ATmega128 보드

AVR 마이크로 컨트롤러 개요

- ATmega128은 Atmel 회사에서 만든 AVR 마이크로 컨트롤러 패밀리의 한 종류
- AVR 마이크로 컨트롤러는 Atmel 회사에 의해서 1997년에 발표
- 2016년에 Microchip Technology에 합병되어 현재는 존재하지 않는 회사

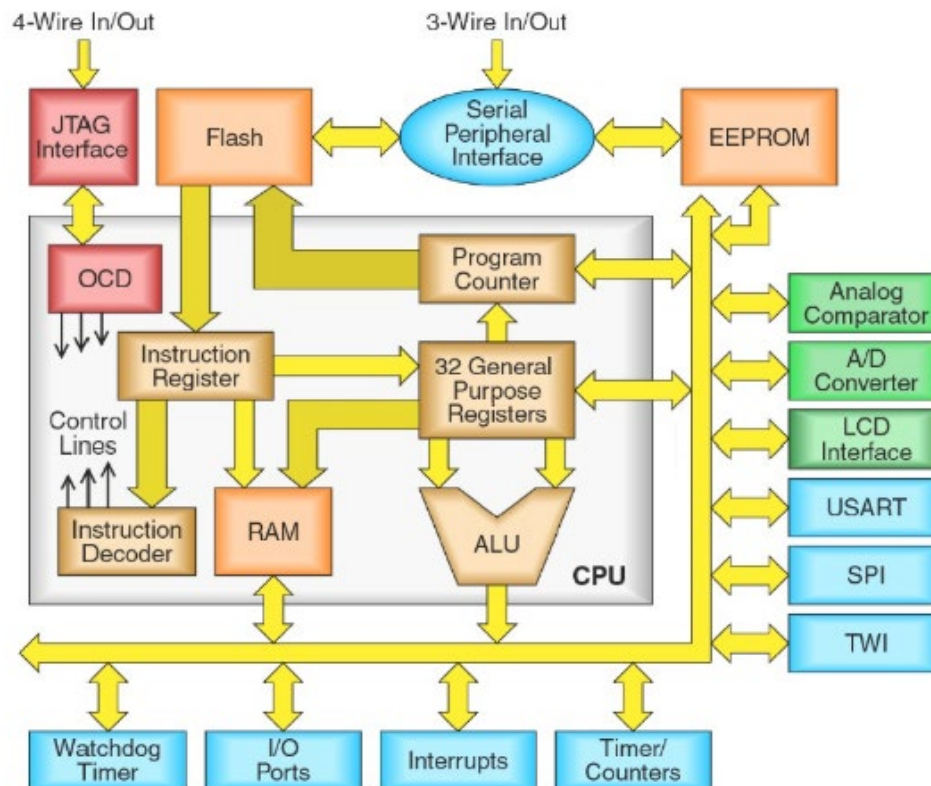
❖ AVR 마이크로 컨트롤러의 대표 특징

- 8비트 마이크로 컨트롤러이므로 데이터 버스가 8비트이지만, 인터랙션 버스는 16비트 버스를 채택해서 빠른 처리가 가능
- RISC 구조이며, 파이프라인 처리 방식을 도입한 고성능 구조
- 한 개의 클럭 사이클 당 한 개의 인스트럭션을 처리할 수 있음
 - ✓ 16MHz의 클럭을 사용할 경우, 1초에 16,000,000개의 인스트럭션을 처리
- 32개의 범용 레지스터(general purpose register)를 가지고 있음
- 다른 마이크로 컨트롤러에 비해서 코드 사이즈가 작음.



AVR 마이크로 컨트롤러 개요

- AVR 마이크로 컨트롤러는 하버드 아키텍처를 채택하여 프로그램 메모리와 데이터 메모리가 분리된 구조로 되어있음
- 프로그램 메모리와 데이터 메모리를 동시에 접근하는 것이 가능



■ AVR 마이크로 컨트롤러 개요

❖ tinyAVR

- ATtiny 시리즈의 마이크로 컨트롤러
- 6~32핀이며, 비교적 작은 크기의 IC에 많지 않은 종류의 주변 장치들을 내장

❖ megaAVR

- ATmega 시리즈의 마이크로 컨트롤러
- 28~100핀, 많은 종류의 주변 장치를 내장하고 있으며, 비교적 큰 용량의 메모리를 내장

❖ xmega

- ATxmega 시리즈의 마이크로 컨트롤러
- 44~100핀이며, ATmega 시리즈에 비해서 많은 종류의 주변 장치와 큰 용량의 메모리를 내장

	타이니	메가	X-메가
프로그램 메모리(KB)	1~8	4~256	32~384
핀 수(개)	6~32	28~100	44~100
지원하는 어셈블리 명령어 수(개)	~120	~135	~142
최대 동작 속도(MHz)	20	20	32

AVR 마이크로 컨트롤러 비교

❖ 8051

- 인텔이 1980년 발표한 8비트 마이크로 컨트롤러
- CISC 구조를 사용
- 다양한 호환 마이크로 컨트롤러가 생산되고 있음

❖ ARM Cortex-M

- ARM의 아키텍처를 사용하고 아트멜에서 제작하는 32비트 마이크로 컨트롤러
- Cortex-M0, Cortex-M+, Cortex-M3, Cortex-M4, Cortex-M7 등
- SAM (Smart ARM-based Microcontroller unit) 시리즈

❖ AVR

- 1996년 아트멜이 발표한 고유의 아키텍처를 사용한 8비트 마이크로 컨트롤러
- ATmega128은 AVR Mega 시리즈에 속함

■ AVR 마이크로 컨트롤러 비교

❖ ATmega128

- 2001년 처음 발표
- 5V에서 16MHz로 동작

❖ ATmega128L

- 저전력 버전
- 2.7V에서 8MHz로 동작

❖ ATmega128A

- 2008년 발표
- ATmega128 + ATmega128L
- 현재 'ATmega128'로 언급되는 마이크로 컨트롤러는 실제로는 'ATmega128A'임

 ATmeag128

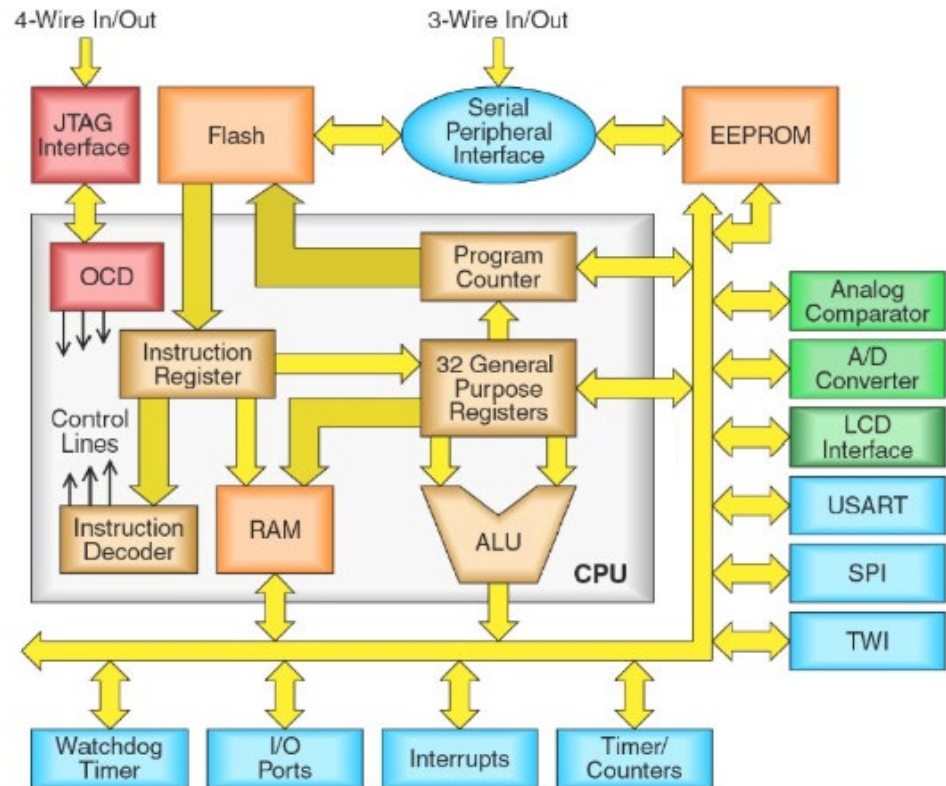
❖ 개선된 RISK 구조

- 다양한 어셈블리 명령어 (하드웨어 수준의 명령어)
- 파이프라인 구조
 - ✓ 대부분의 명령어는 1클럭 소요 : 16MIPS
 - ✓ 2클럭을 필요로 하는 곱셈기 제공
- 32개의 8비트 범용 레지스터와 I/O 레지스터 제공
- 정적 동작 지원
 - ✓ SRAM으로 구성된 메모리
 - ✓ 클럭 변경에 따른 리프레시 오류 없음

ATmega128

❖ 다양한 메모리

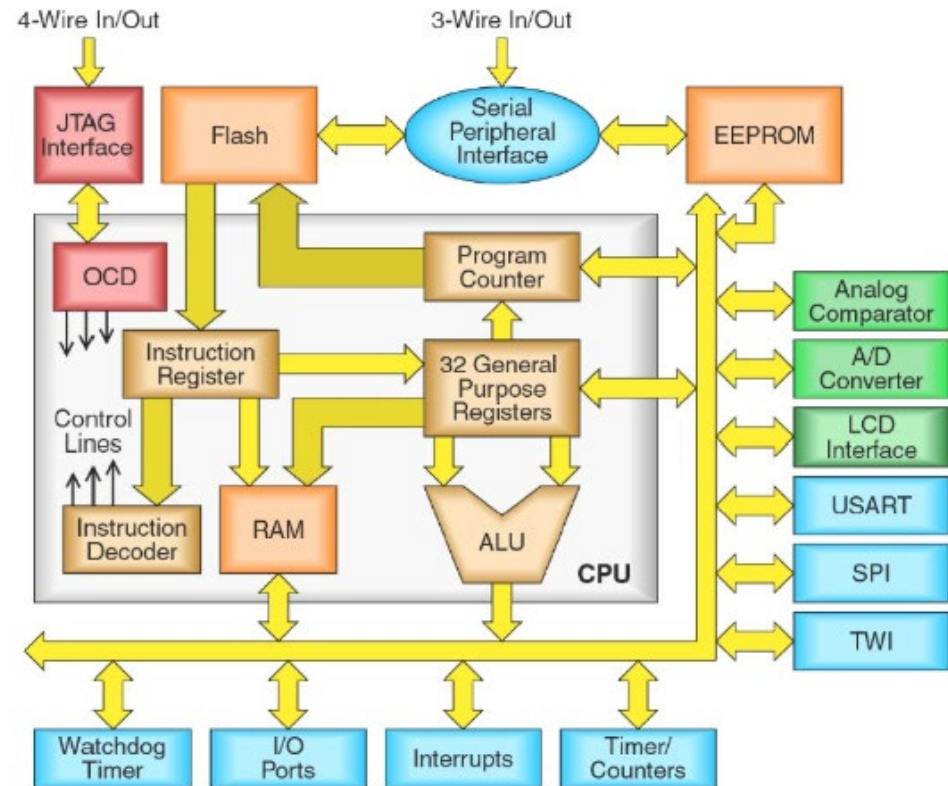
- 128KB 비휘발성 플래시 메모리
 - ✓ 프로그램 메모리
 - ✓ 10,000회 쓰기 보장
- 4KB 비휘발성 EEPROM
 - ✓ 데이터 메모리
 - ✓ 100,000회 쓰기 보장
- 4KB 휘발성 SRAM
 - ✓ 데이터 메모리
- 최대 64KB 외부 메모리 연결 가능
- ISP 방식의 프로그래밍



ATmeag128

❖ 다양한 주변장치

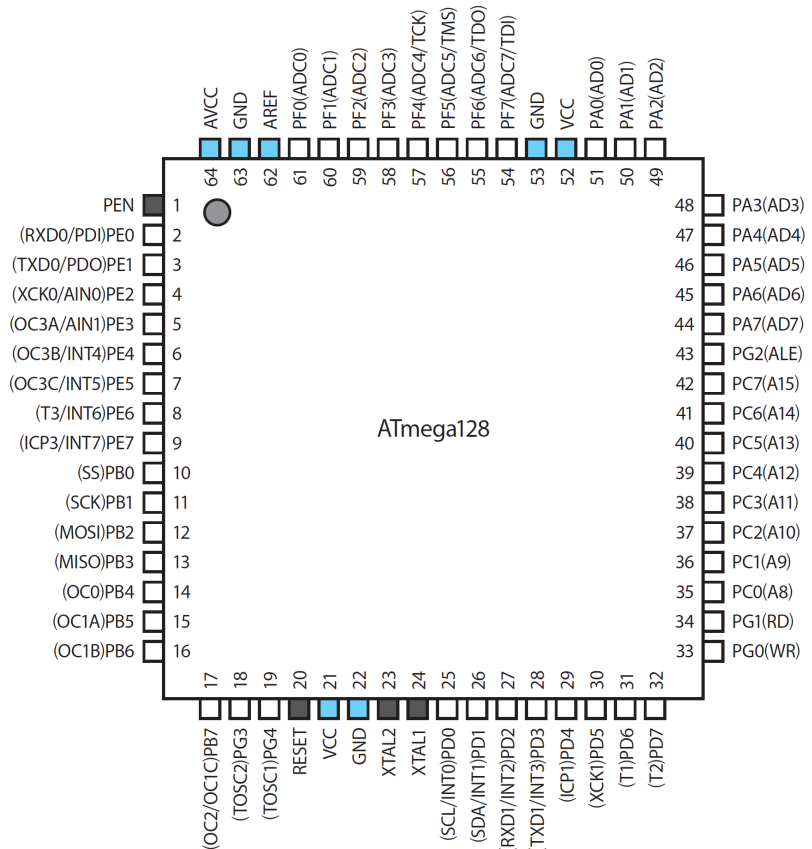
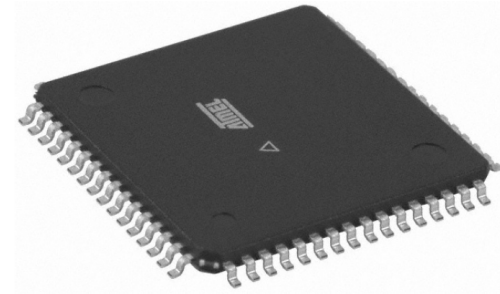
- 2개의 8비트 타이머/카운터
- 2개의 16비트 타이머/카운터
- 독립된 오실레이터로 동작하는 실시간 카운터(RTC)
- 다양한 해상도의 PWM 채널
- 8채널 10비트 ADC
- TWI 통신 지원
- 2개의 USART 통신 지원
- SPI 통신 지원
- 워치도그 타이머
- 아날로그 비교기



ATmega128

❖ 기타 특징

- 브라운아웃 감지
- 내부 오실레이터로 외부 클럭 없이 동작 가능
- 다양한 인터럽트
- 전력 제어를 위한 6가지 슬립 모드
- 64핀
- TQFP 패키지가 가장 흔함
- 동작 전압 : 2.7~5.5V
- 동작 속도 : 0~16MHz



ATmeag128

❖ Register

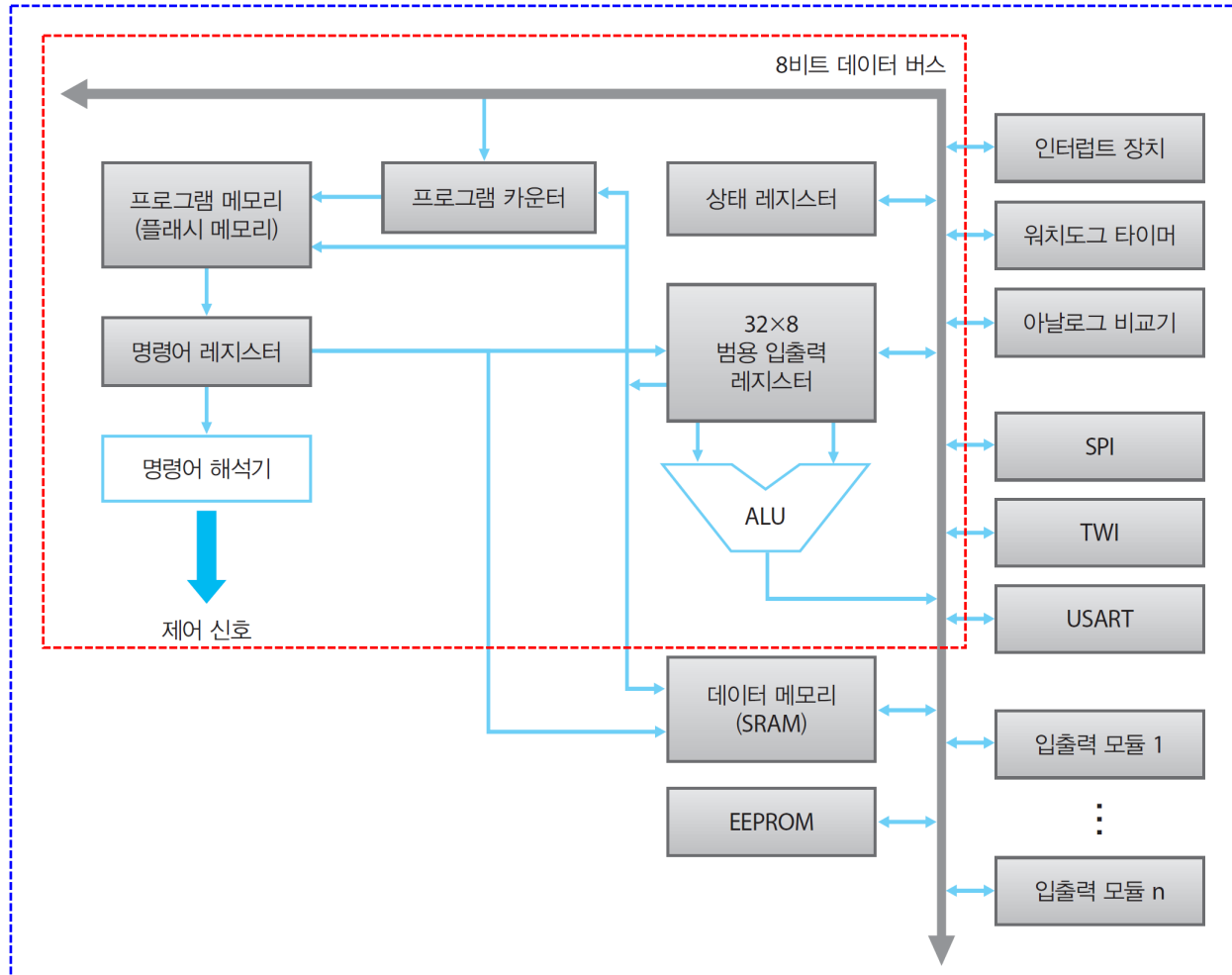
- CPU 내부의 레지스터
 - ✓ 마이크로프로세서에서 사용되는 레지스터
 - ✓ 연산 직전에 데이터와 명령어를 저장하는 범용 레지스터와 연산 상태 등을 저장하는 전용 레지스터로 구성
 - ✓ 프로그래머가 신경 쓰지 않아도 되는 레지스터
- CPU 외부의 레지스터
 - ✓ 외부 장치와 데이터 교환 과정에서 필요한 레지스터
 - ✓ 입출력 레지스터
 - ✓ 이름이 정해져 있으므로 미리 정의된 변수와 유사함
 - ❖ 번지로도 사용이 가능하지만 흔히 사용하지는 않음
 - ❖ io.h (iom128a.h) 파일에 이름이 정의되어 있음

2

ATmega128 보드

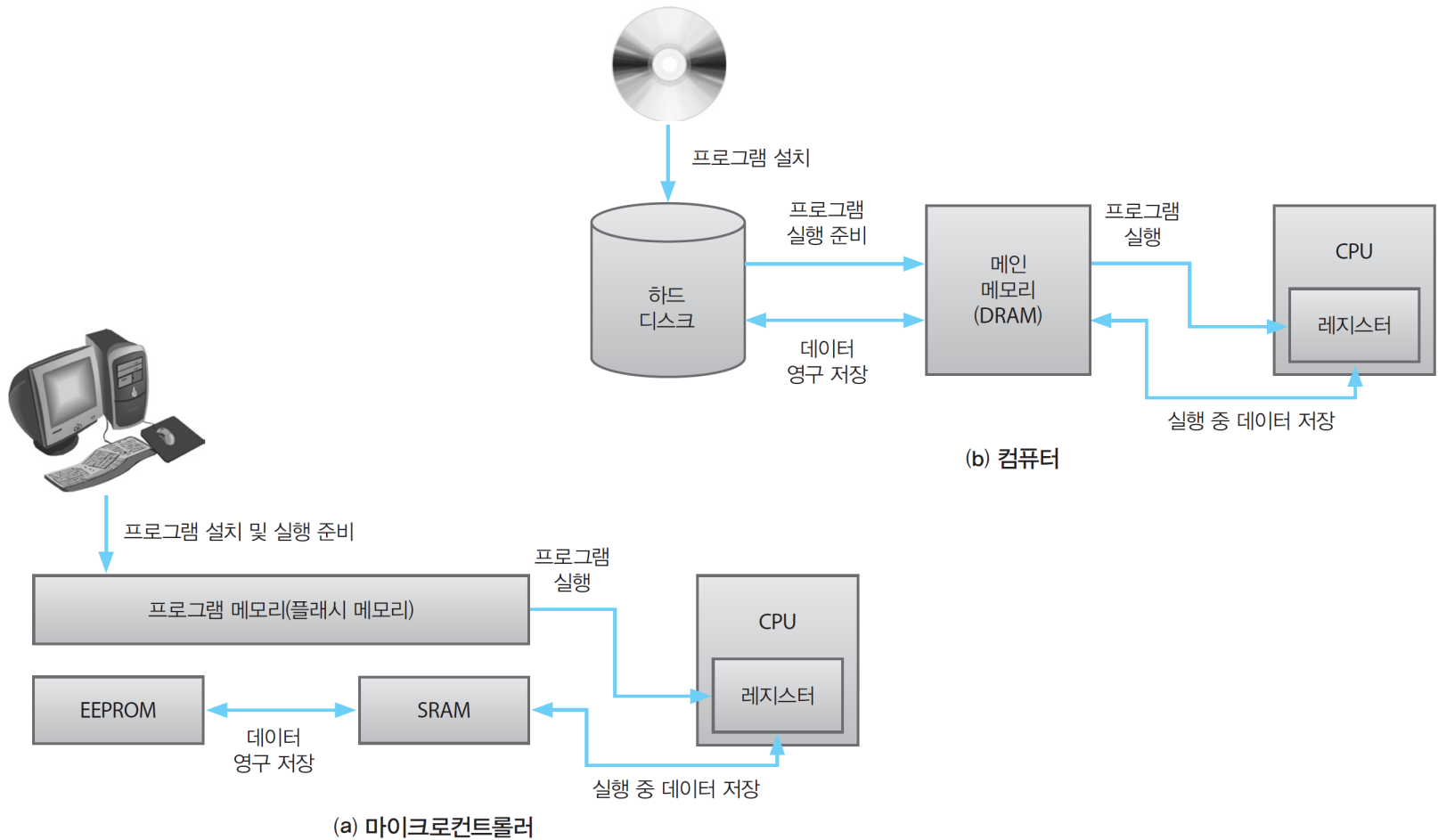
ATmeag128 내부 구조

마이크로 프로세서



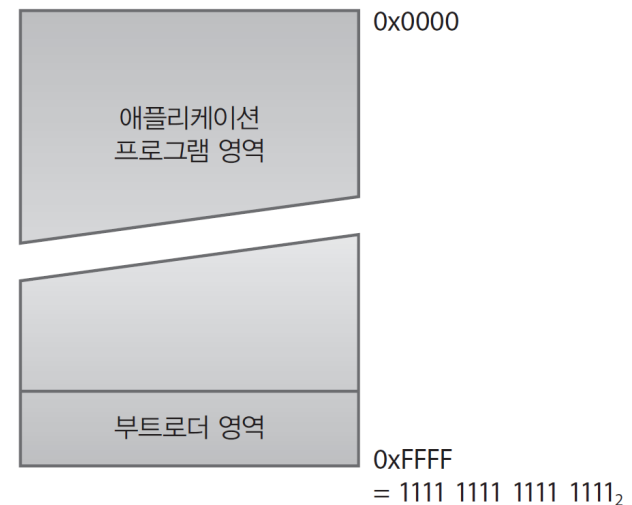
마이크로 컨트롤러

■ 프로그램 실행 과정과 메모리



메모리 주소 – 플래시 메모리

- 2바이트 단위로 명령어를 저장
 - ✓ 128KByte 메모리를 2바이트 단위로 번지 지정
 - ✓ 16비트의 0x0000 ~ 0xFFFF 번지 사용
- 애플리케이션 프로그램 영역 + 부트로더 영역
 - ✓ 애플리케이션 프로그램 영역 : 사용자 프로그램 저장
 - ✓ 부트로더 영역 : 특수 목적용 프로그램 저장



메모리 주소 – SRAM

- 256개의 레지스터 역시 SRAM과 동일한 메모리 주소 공간을 사용
 - ✓ 레지스터와 SRAM을 '데이터 메모리'로 함께 지칭
 - ✓ 16비트 주소 필요



	사용 가능한 개수	실제 정의되어 사용하는 개수	추후 사용을 위해 예약된 개수
범용 레지스터	32	32	0
입출력 레지스터	64	64	0
확장 입출력 레지스터	160	41	119
합계	256	137	119

■ 메모리 주소 – EEPROM

- 독립된 메모리 주소 공간을 가짐
- 데이터 메모리의 일종이지만 독립된 메모리 주소 공간을 가지므로 데이터 메모리로 분류하지 않는 경우가 많음
- 번지로만 접근이 가능한 유일한 메모리임
 - ✓ 플래시 메모리 : 사용자가 임의로 프로그램이 저장되는 번지를 지정할 수 없음
 - ✓ SRAM : CPU의 연산 결과에 따라 자동으로 값이 저장되므로 역시 임의로 번지를 지정할 수 없음
 - ✓ 레지스터 : 번지로 접근이 가능하지만, 기억하기 쉬운 레지스터 이름으로 접근하는 것이 일반적임

■ 전용 레지스터 – 상태 레지스터

❖ CPU의 연산 결과를 반영하는 전용 레지스터

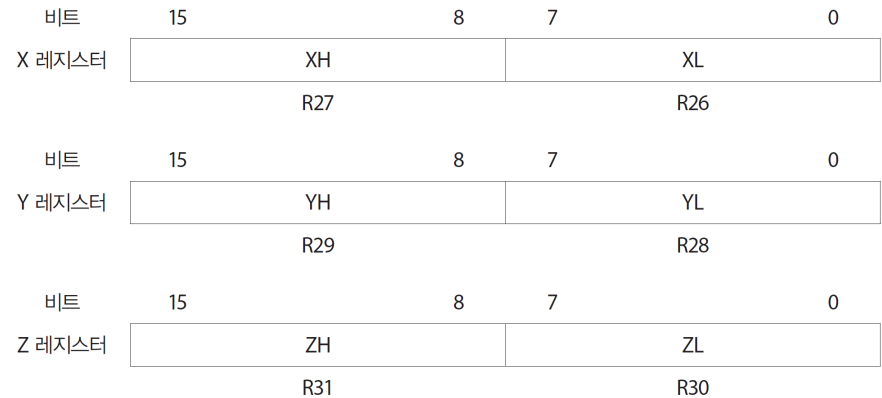
비트	7	6	5	4	3	2	1	0
비트 이름	I	T	H	S	V	N	Z	C
읽기/쓰기	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
초깃값	0	0	0	0	0	0	0	0

비트 번호	비트 이름	설명
7	I	Global Interrupt Enable: 전역적인 인터럽트 발생을 허용한다. 인터럽트 발생을 위해서는 반드시 전역 인터럽트 비트가 설정되어 있어야 하며, SEI(Set Interrupt), CLI(Clear Interrupt) 명령을 통해 설정할 수 있다.
6	T	Bit Copy Storage: 비트 복사를 위한 BLD(Bit Load), BST(Bit Store) 명령에서 사용한다. BST 명령에 의해 비트 값을 비트 T에 저장할 수 있으며, BLD 명령에 의해 비트 T의 내용을 읽어 올 수 있다.
5	H	Half Carry Flag: 산술 연산에서의 보조 캐리 발생을 나타낸다. 보조 캐리는 바이트 단위 연산에서 하위 니블(nibble)로부터 발생하는 자리 올림을 말한다.
4	S	Sign Bit: 부호 비트로, 음수 플래그(N)와 2의 보수 오버플로 플래그(V)의 배타적 논리합(XOR)으로 설정된다($S = N \oplus V$).
3	V	2's Complement Overflow Flag: 2의 보수를 이용한 연산에서 자리 올림이 발생했음을 나타낸다.
2	N	Negative Flag: 산술 연산이나 논리 연산에서 결과가 음수임을 나타낸다.
1	Z	Zero Flag: 산술 연산이나 논리 연산에서 결과가 영(zero)임을 나타낸다.
0	C	Carry Flag: 산술 연산이나 논리 연산에서 캐리가 발생했음을 나타낸다.

범용 레지스터

- 32개의 범용 레지스터 : 연산 과정에서 필요한 임시 결과 저장용
- 16비트 값 저장을 위해 2개의 8비트 범용 레지스터를 조합해서 사용
 - ✓ 메모리의 주소는 16비트가 필요

MSB	LSB	
R0	0x00	
R1	0x01	
R2	0x02	
R3	0x03	
...		
R26	0x1A	X 레지스터 하위 바이트
R27	0x1B	X 레지스터 상위 바이트
R28	0x1C	Y 레지스터 하위 바이트
R29	0x1D	Y 레지스터 상위 바이트
R30	0x1E	Z 레지스터 하위 바이트
R31	0x1F	Z 레지스터 상위 바이트



■ 시스템 클럭

- 클럭 : 디지털 시스템에서 동작의 기준이 되는 신호
 - ✓ ATmega128을 위해서는 16MHz 클럭이 주로 사용
- 시스템 클럭 : CPU의 연산 기준
- 기타 클럭 : 시스템 클럭 이외의 클럭
 - ✓ ADC를 위한 클럭
 - ❖ 아날로그 값을 디지털 값으로 변환하는 과정에서 사용되는 클럭으로 시스템 클럭보다 낮음
 - ❖ 시스템 클럭을 분주(prescale)하여 사용
 - ✓ 위치도그를 위한 클럭
 - ❖ 시스템의 오류 발생 검사를 위해 사용
 - ❖ 시스템 클럭과는 별도의 내부 클럭 사용
 - ✓ RTC 클럭
 - ❖ 시계 기능을 위한 외부의 별도 클럭 사용

클럭 소스

- 크리스탈
 - ✓ ATmega128을 위한 클럭 소스로 흔히 사용
 - ✓ 무극성의 소자로 수정(크리스탈)이 전기 신호에 따라 진동하는 성질 사용
 - ✓ 외부 회로가 필요하며 ATmega128의 내부 회로와 함께 사용
- 레조네이터
 - ✓ 세라믹으로 만들어진 클럭 생성 소자
 - ✓ 압전 효과를 이용하여 클럭 생성
 - ✓ 안정도가 크리스탈에 비해 떨어짐
- 오실레이터
 - ✓ 외부 회로 없이 자체적으로 클럭 생성
 - ✓ 전원 연결을 위한 핀 필요



(a) 크리스탈



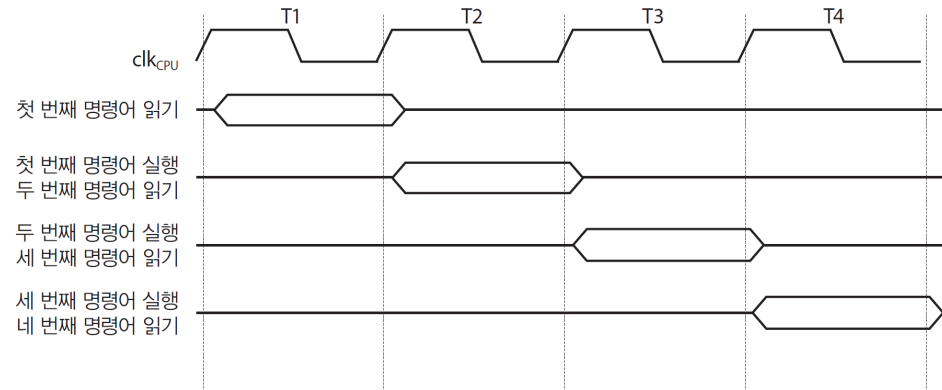
(b) 오실레이터

명령어(Instruction) 실행

- 명령어 읽기와 명령어 실행의 2단계가 파이프라인에 의해 중첩되어 실행

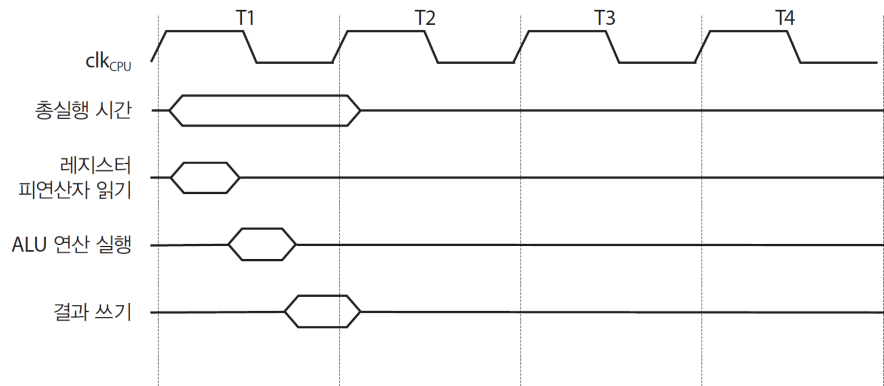
✓ 1클록에 1개 명령어 실행 가능

✓ RISC 구조이므로 가능



- CPU 내에서의 명령어 실행은 3단계의 세부 단계로 나뉘어 진행

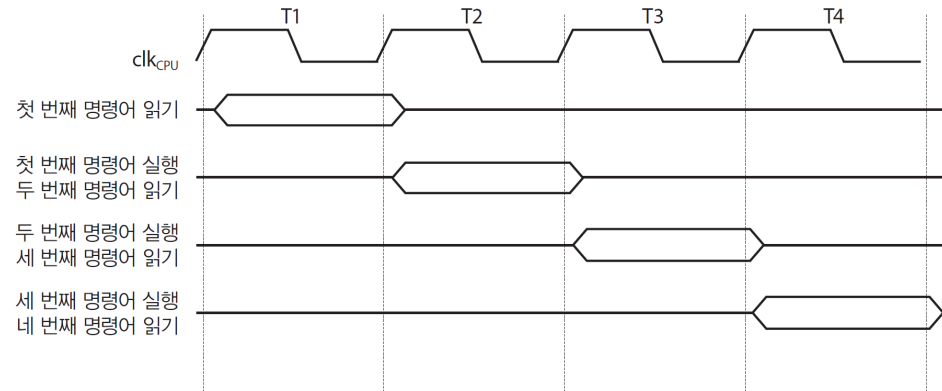
• 읽기 → ALU 내의 연산 → 결과 쓰기



명령어(Instruction) 실행

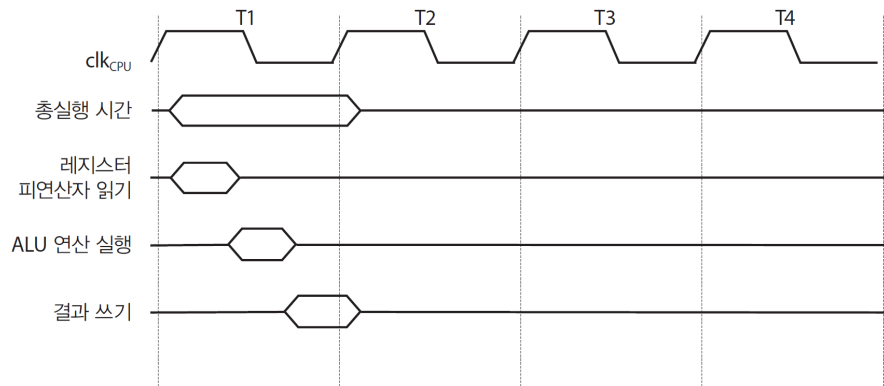
- 명령어 읽기와 명령어 실행의 2단계가 파이프라인에 의해 중첩되어 실행

- ✓ 1클록에 1개 명령어 실행 가능
- ✓ RISC 구조이므로 가능

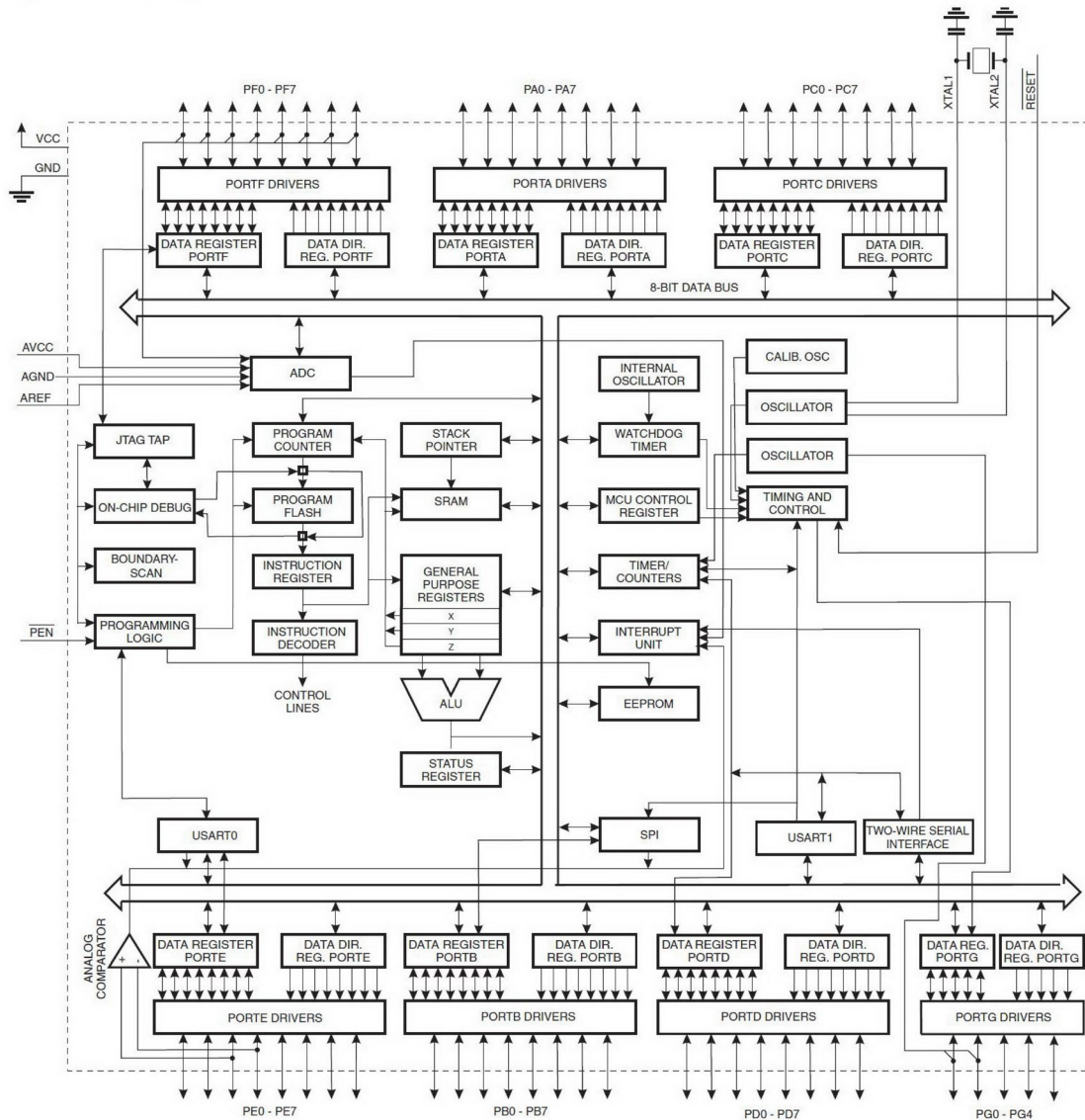


- CPU 내에서의 명령어 실행은 3단계의 세부 단계로 나뉘어 진행

- 읽기 → ALU 내의 연산 → 결과 쓰기



ATmega128 마이크로 컨트롤러 내부 구조



■ ATmega128 마이크로 컨트롤러 Lock bit

- 메모리 보호를 위한 1바이트의 락 비트(lock bit)
- 마이크로 컨트롤러에 내장된 플래쉬 메모리와 EEPROM에 대해서 읽기 혹은 쓰기 금지(lock) 등을 설정하기 위한 비트
- 기본 값들은 모두 1(unprogrammed)로 설정 되어, 읽기 혹은 쓰기 금지(lock) 기능이 설정되어 있지 않은 상태
- 읽기 혹은 쓰기 금지 기능을 설정하기 위해서는 필요한 비트들의 값을 0(programmed)으로 변경

비트	비트 이름	기능	기본값
7			
6			
5	BLB12	Boot lock bit	1(unprogrammed)
4	BLB11	Boot lock bit	1(unprogrammed)
3	BLB02	Boot lock bit	1(unprogrammed)
2	BLB01	Boot lock bit	1(unprogrammed)
1	LB2	Lock bit	1(unprogrammed)
0	LB1	Lock bit	1(unprogrammed)

ATmega128 마이크로 컨트롤러 Fuse bit

- 퓨즈 비트는 3개 바이트의 비트들로 설정
- Extended Fuse Byte, Fuse High Byte, Fuse Low Byte

Extended Fuse Byte

비트	비트 이름	기능	기본값
7			
6			
5			
4			
3			
2			
1	M103C	ATmega103 호환 모드	0(programmed)
0	WDTON	Watchdog 타이머 활성화	1(unprogrammed)

Fuse High Byte

비트	비트 이름	기능	기본값
7	OCDEN	OCD(On-chip debug) 활성화	1(unprogrammed)
6	JTAGEN	JTAG 기능 활성화	0(programmed)
5	SPIEN	SPI 직렬 프로그래밍 활성화	0(programmed)
4	CKOPT	오실레이터 옵션	1(unprogrammed)
3	EESAVE	Chip erase시 EEPROM 내용 보존	1(unprogrammed)
2	BOOTSZ1	Boot size 선택	0(programmed)
1	BOOTSZ0	Boot size 선택	0(programmed)
0	BOOTRST	Reset vector 선택	1(unprogrammed)

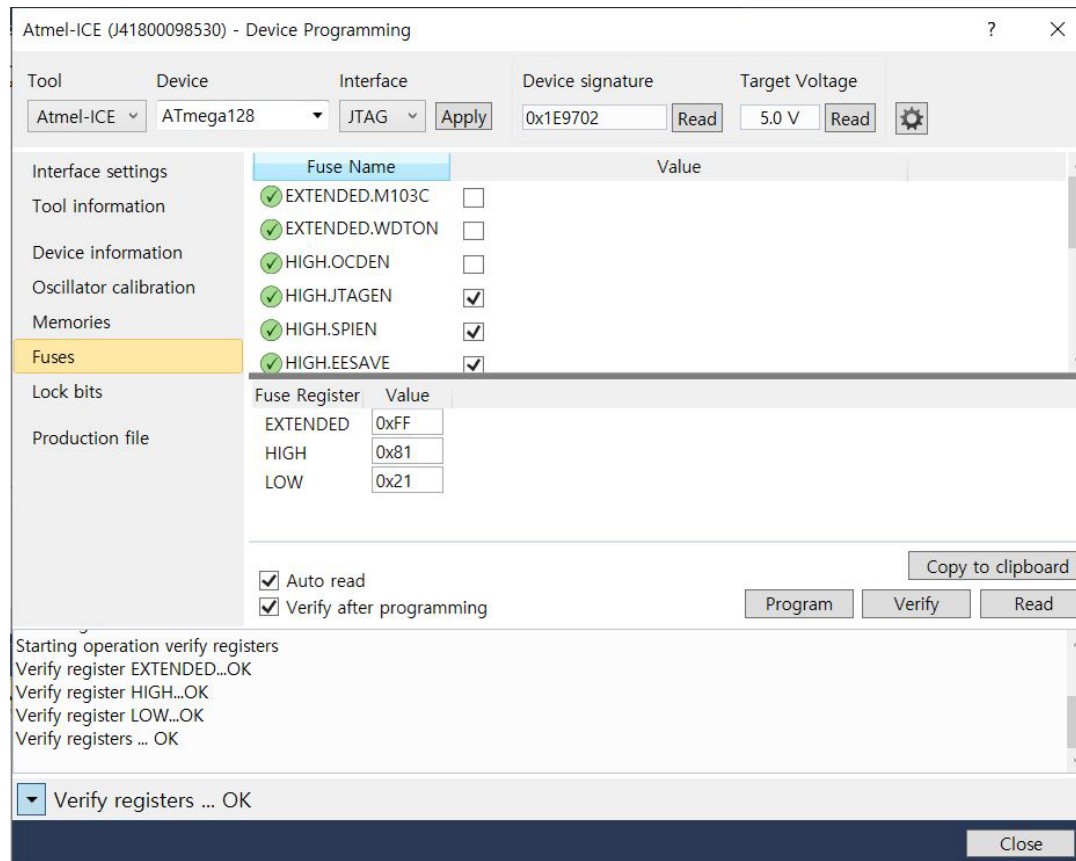
Fuse Low Byte

비트	비트 이름	기능	기본값
7	BODLEVEL	Brown out 감지 트리거 레벨	1(unprogrammed)
6	BODEN	Brown out 감지 활성화	1(unprogrammed)
5	SUT1	Start-up time 선택	1(unprogrammed)
4	SUT0	Start-up time 선택	0(programmed)
3	CKSEL3	클럭 소스 선택	0(programmed)
2	CKSEL2	클럭 소스 선택	0(programmed)
1	CKSEL1	클럭 소스 선택	0(programmed)
0	CKSEL0	클럭 소스 선택	1(unprogrammed)

- SPIEN은 SPI 시리얼 프로그램을 이용해서 프로그램 메모리와 EEPROM 메모리의 프로그래밍을 가능하게 해주는 비트
- ISP를 이용해서 프로그램을 다운로드하기 위해서는 기본 값인 0(programmed)으로 설정
- SPIEN은 SPI 시리얼 프로그램을 이용해서 프로그램 메모리와 EEPROM 메모리의 프로그래밍을 가능하게 해주는 비트
- ISP를 이용해서 프로그램을 다운로드하기 위해서는 기본 값인 0(programmed)으로 설정

ATmega128 마이크로 컨트롤러 Fuse bit

- Microchip Studio에서 설정





Q & A
